FUNCTIONAL & TECHNICAL SPECIFICATION

Shopify Store Migration App

Move products, customers, orders and content from WooCommerce, WordPress and other platforms into Shopify — screens, process, database structure and requirements.

Document version	1.0
Date	6 October 2026
Codebase	Node.js (Express, EJS) · MySQL · Shopify Admin GraphQL API
Built on	Existing app skeleton: install flow, setup, plans, help & support
Phase 1 sources	WooCommerce, WordPress

Contents

1. Overview and scope
2. App navigation
3. Dashboard screen — migration cards
4. Migrations screen — choose, connect and begin
5. Progress and results screens
6. How the migration process works
7. Credentials the merchant must provide
8. Field mapping — WooCommerce to Shopify
9. Database structure
10. Shopify requirements
11. Routes and endpoints
12. Project file structure
13. Plans, limits and demo migration
14. What cannot be migrated
15. Build phases
16. Appendix A — Full SQL

1. Overview and scope

The app is an embedded Shopify app. A merchant installs it on their **new Shopify store**, connects their **old store** (WooCommerce, WordPress, etc.) by entering its address and API keys, ticks the data they want to bring across, and starts the migration. The work runs on the server in the background, so the merchant can close the browser and come back. When it finishes they see what moved, what failed and why, and can retry the failures.

1.1 Supported sources

Platform	Phase	Connection method
WooCommerce	Phase 1	WooCommerce REST API v3 — Consumer Key + Consumer Secret
WordPress	Phase 1	WordPress REST API — public content needs only the site URL; private content needs a username + Application Password
CSV file (any platform)	Phase 2	Upload of a CSV in the app's template format
Magento 2 / Adobe Commerce	Phase 3	REST API — Integration access token
BigCommerce	Phase 3	REST API — Store hash + API access token
PrestaShop, OpenCart, Wix, Squarespace	Later	API where available, otherwise CSV

1.2 What can be migrated

Data	From WooCommerce / WordPress	Becomes in Shopify
Products	Simple & variable products, variants, SKUs, prices, stock, images, tags, weight	Products with variants, media and inventory
Categories	Product categories	Custom collections (products linked)
Customers	Name, email, phone, addresses, marketing consent	Customers (invited to set a new password)
Orders	Historical orders, line items, totals, tax, shipping, status, notes	Orders (historical, no emails sent)
Coupons	Coupon codes (percent / fixed)	Discount codes
Pages	WordPress pages	Online Store pages
Blog posts	WordPress posts	Blog articles
SEO redirects	Old product / category / page URLs	301 URL redirects

2. App navigation

The app nav (left menu inside Shopify admin) has four entries:

Menu item	Route	Purpose
Dashboard	`/dashboard`	Summary cards: how many migrations, how many records moved, what is running now; recent migrations list.
Migrations	`/migrations`	List of all migrations and the **New migration** wizard where the merchant chooses the platform, enters the keys, picks what to migrate and begins the process.
Plans	`/plans`	Existing screen. Plan cards, Shopify billing. Plans gain record limits (section 13).
Help & support	`/help`	Existing screen. Installation steps, how to create API keys, FAQ, contact.

```
<s-app-nav>
  <s-link href="/dashboard" rel="home">Dashboard</s-link>
  <s-link href="/migrations">Migrations</s-link>
  <s-link href="/plans">Plans</s-link>
  <s-link href="/help">Help &amp;amp; support</s-link>
</s-app-nav>
```

2.1 Setup steps (first open after install)

The existing setup screen gets three steps, each worked out from the store's own data:

#	Step	Done when
1	Connect your old store	A row exists in `source_connections` with `status = 'connected'`
2	Run a free demo migration	A `migrations` row with `type = 'demo'` and `status = 'completed'`
3	Choose a plan	An active row in `user_memberships`

3. Dashboard screen — migration cards

The dashboard answers three questions at a glance: **how many migrations have I run, how much data has moved, and is anything running or failing right now.**

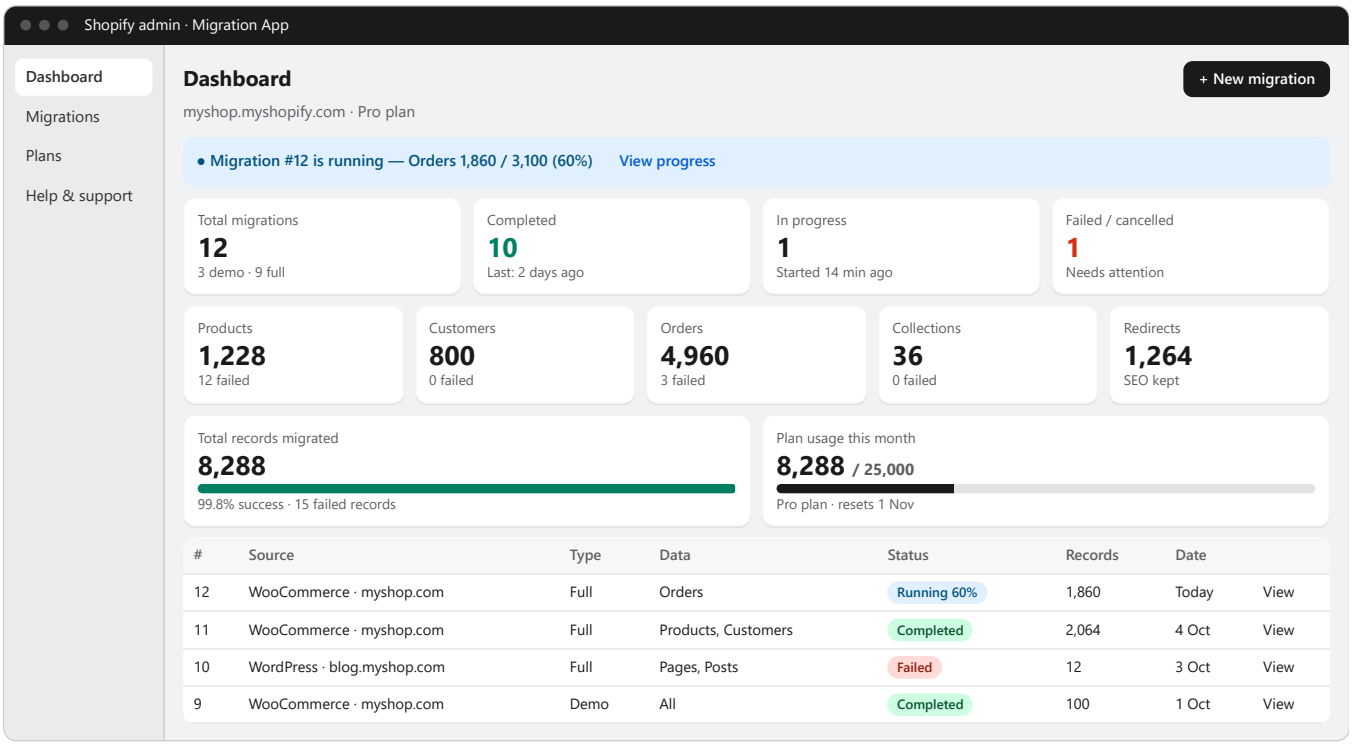


Figure 1 — Dashboard with summary cards, per-data cards and recent migrations.

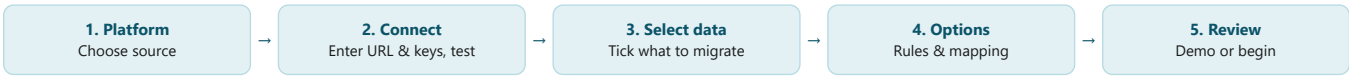
3.1 Cards and where each number comes from

Card	Query (all scoped to `store_id = ?`)
Total migrations	`SELECT COUNT(*), SUM(type='demo'), SUM(type<>'demo') FROM migrations WHERE status <> 'draft'`
Completed	`COUNT(*) … WHERE status = 'completed'` + `MAX(finished_at)`
In progress	`COUNT(*) … WHERE status IN ('queued','running','paused')`
Failed / cancelled	`COUNT(*) … WHERE status IN ('failed','cancelled')`
Per data type (Products, Customers…)	`SELECT s.entity, SUM(s.succeeded), SUM(s.failed) FROM migration_steps s JOIN migrations m ON m.id = s.migration_id WHERE m.store_id = ? AND`
Total records migrated	Sum of the row above; success rate = succeeded ÷ (succeeded + failed)
Plan usage	Succeeded records of non-demo migrations finished in the current billing period vs. the plan's `max_records`
Running banner	The newest migration with `status = 'running'` and its current step
Recent migrations	Last 5 rows of `migrations` joined to `source_connections`, newest first

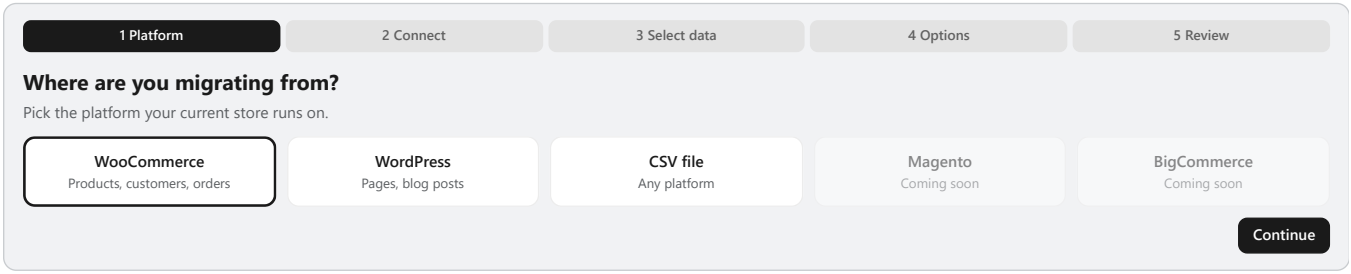
> An empty state is shown before the first migration: "No migrations yet — connect your old store to begin" with a **Start migration** button.

4. Migrations screen — choose, connect and begin

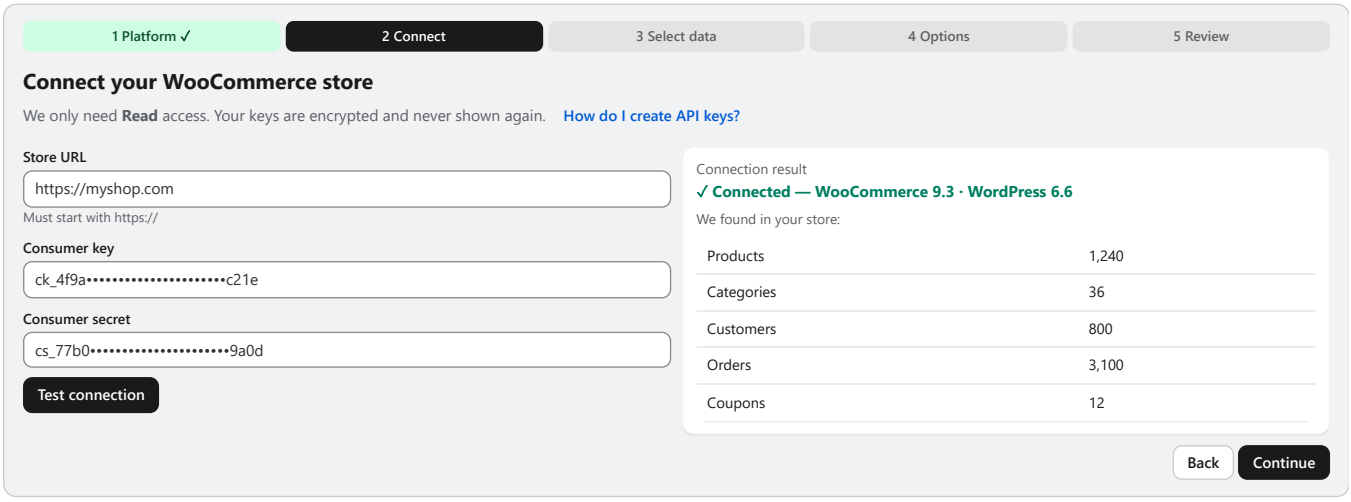
The **Migrations** menu opens a list of all migrations (same table as the dashboard, with filters and paging) and a **New migration** button. New migration is a five-step wizard. Each step is saved as the merchant goes, so leaving and returning resumes where they left off (the migration is stored with `status = 'draft'`).



Step 1 — Choose platform

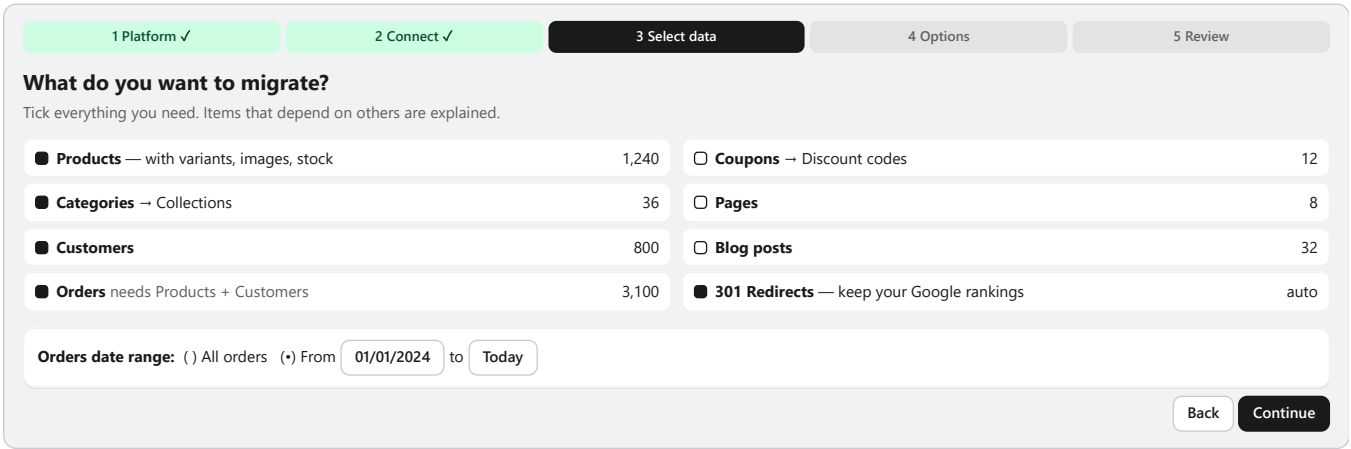


Step 2 — Connect (enter the keys)



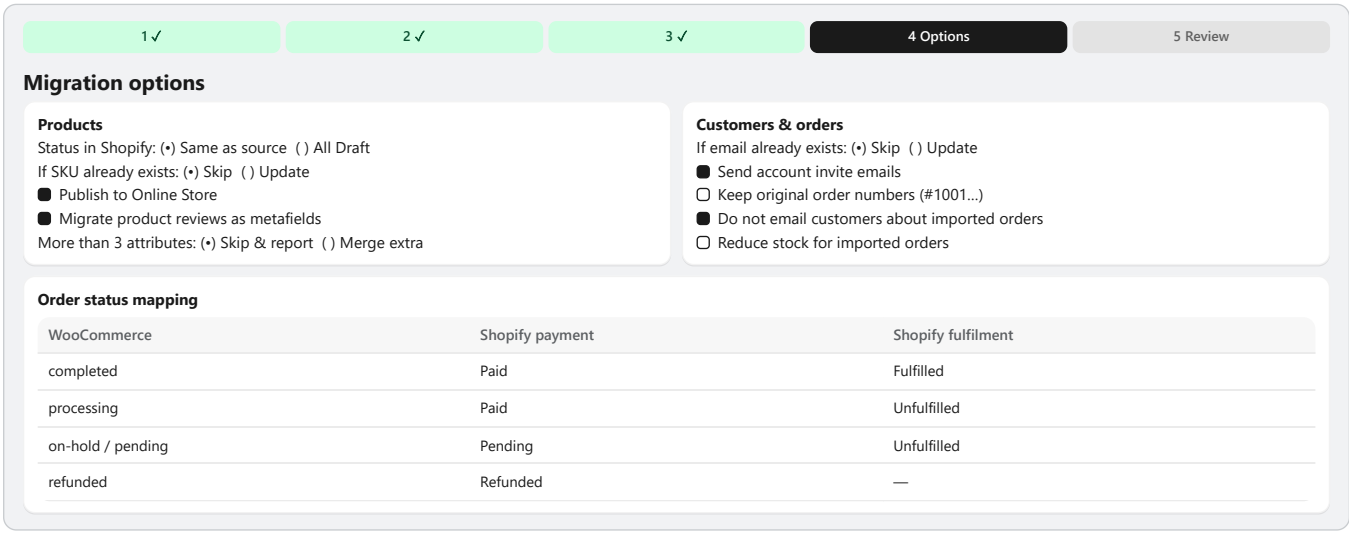
Test connection calls the source API, checks the keys work and have read access, and fetches counts for every data type (WooCommerce returns the total in the `X-WP-Total` header of a `per_page=1` request, so counting is cheap). Errors are shown in plain words: "Keys are wrong", "Keys are Write-only — give them Read access", "Site blocks the REST API", "Site is not HTTPS".

Step 3 — Select what to migrate

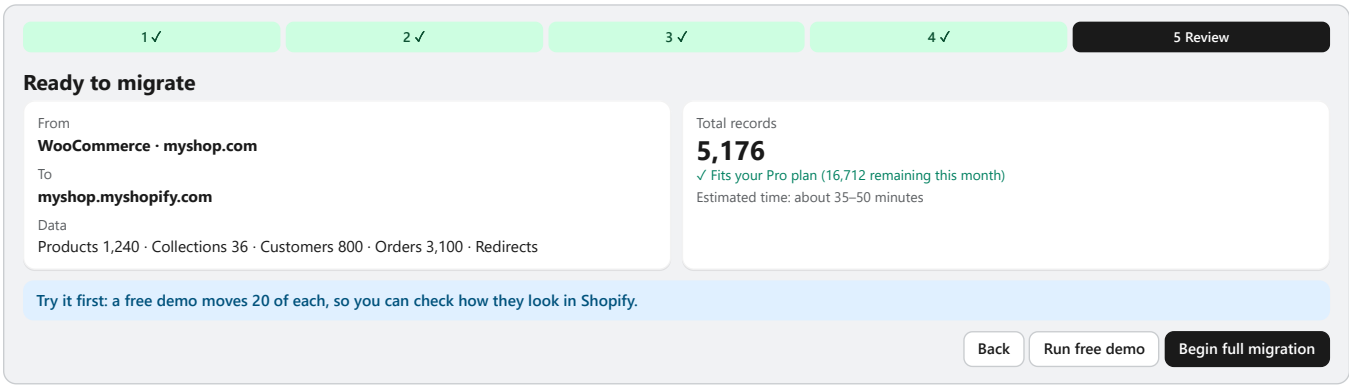


Ticking **Orders** without Products or Customers shows a warning: orders will still import, but their line items and customers will not be linked to Shopify records.

Step 4 — Options



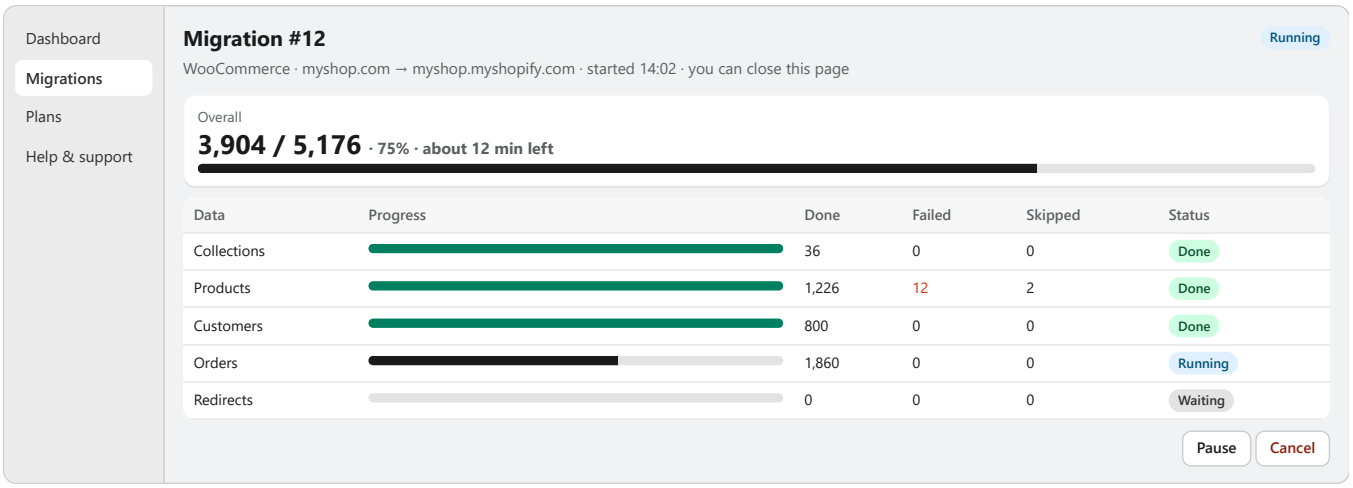
Step 5 — Review and begin



If the selection exceeds the plan, **Begin full migration** is replaced by **Upgrade plan**, which opens the Plans screen. Pressing Begin sets the migration to `queued`, creates one `migration_steps` row per selected data type, and opens the progress screen.

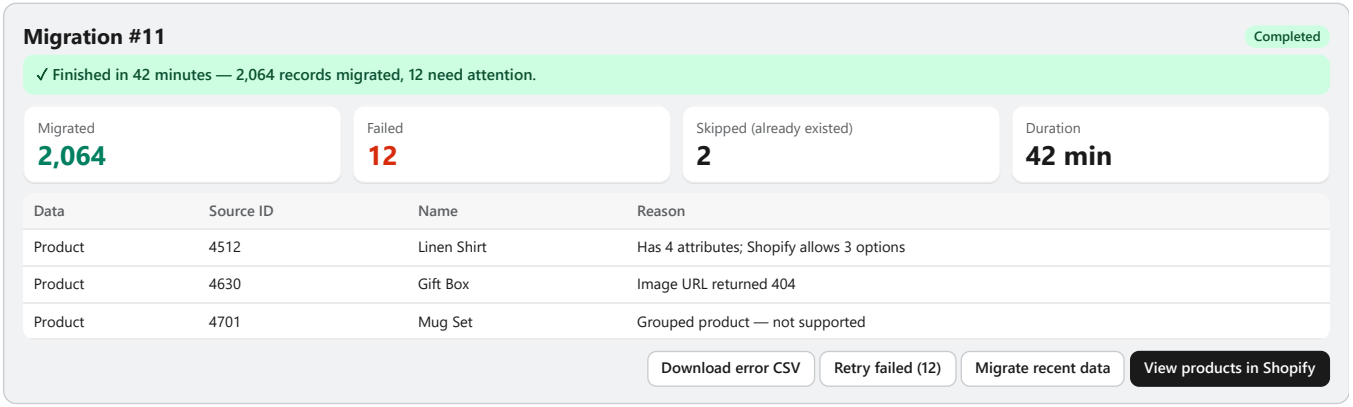
5. Progress and results screens

5.1 Progress — `/migrations/:id` while running



The page polls `GET /migrations/:id/status` every 3 seconds and redraws the bars. Pause stops after the current batch; Resume continues from the saved cursor. Cancel stops the job — records already created stay in Shopify.

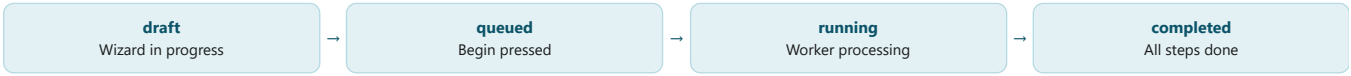
5.2 Results — when finished



- **Retry failed** re-runs only rows in `migration_errors` with `resolved = 0`.
- **Migrate recent data** (delta) creates a new migration of `type = 'delta'` that brings over only records created or changed in the source since this migration — the `id_map` table prevents duplicates.
- **Delete demo data** (on demo migrations only) removes from Shopify every record the demo created, using `id_map.migration_id`.

6. How the migration process works

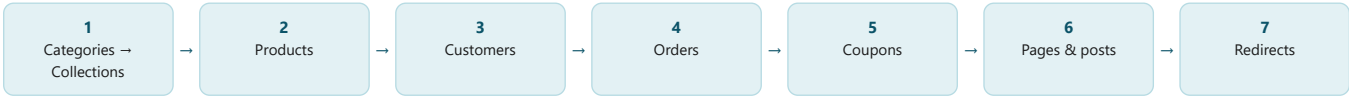
6.1 Lifecycle of a migration



From **running** the merchant can go to **paused** (and back), or **cancelled**. A fatal error (e.g. source keys revoked, Shopify app uninstalled) sets **failed**; it can be resumed once fixed.

6.2 Order of data (dependencies)

Steps always run in this order, because later data links to earlier data through `id_map`:



6.3 The background worker

A worker loop runs inside the Node process (later: separate worker process / BullMQ queue). Every few seconds:

1. **Claim** one migration in `queued` or `running` whose lock is free or stale: `UPDATE migrations SET locked_at = NOW(), worker_id = ? WHERE id = ? AND (locked_at IS NULL OR locked_at < NOW() - INTERVAL 2 MINUTE)`. Only one worker works a migration at a time; a crashed worker's lock expires and another resumes it.
2. **Pick the step** — the first `migration_steps` row (by `position`) not yet done.
3. **Fetch a page** from the source using the step's saved `cursor` (e.g. WooCommerce `page=7&per_page=50`).
4. **For each record:** look it up in `id_map` → if it already exists, skip or update per options; otherwise **transform** it to Shopify input (replacing source IDs with Shopify IDs from `id_map`) and **create** it via GraphQL.
5. **Record the result**: success → insert `id_map` row, `succeeded + 1`; error → insert `migration_errors` row, `failed + 1`. One bad record never stops the migration.
6. **Save the cursor** and counters after every page, and refresh `locked_at` (heartbeat). A restart therefore loses at most one page of work — and `id_map` stops that page being duplicated.
7. When the source returns no more records, mark the step **done** and move on. When every step is done, mark the migration **completed**, set `finished_at`, and (optionally) email the merchant.

6.4 Source adapters

Each platform is one file implementing the same interface, so adding Magento later is adding one file:

```
// services/sources/woocommerce.js
module.exports = {
  platform: "woocommerce",
  credentials: ["base_url", "consumer_key", "consumer_secret"],   // drives the Connect form
  entities: ["categories", "products", "customers", "orders", "coupons"],

  async test(conn)                    { /* → { ok, version, error } */ },
  async count(conn, entity, filters)  { /* → number (X-WP-Total header) */ },
  async fetchPage(conn, entity, cursor, filters)
                                      { /* → { items: [...], nextCursor | null } */ },
  transform(entity, item, ctx)        { /* → Shopify GraphQL input; ctx.idMap, ctx.options */ },
};
```

The Shopify side is shared by all sources: `services/destinations/shopify.js` has one `create<Entity>()` function per data type, built on the existing `services/shopify.js` client (which already handles token refresh, throttling and retries).

6.5 Rate limits and speed

- **Shopify** limits GraphQL by query cost (a leaky bucket that refills every second). Read `extensions.cost.throttleStatus` on each response and slow down when the bucket runs low. The existing client already backs off on `THROTTLED`.
- **Source sites** are often on cheap hosting. Fetch 50–100 records per page, at most 2 requests in parallel, 30-second timeout, retry 3× with backoff.
- **Large stores (10,000+ products):** switch to Shopify **bulk operations** — write the inputs to a JSONL file, upload it with `stagedUploadsCreate`, run `bulkOperationRunMutation`, then read back the result file to fill `id_map`.
- **Images** are passed to Shopify as public URLs (`originalSource`); Shopify downloads them itself. No image passes through our server.

7. Credentials the merchant must provide

Platform	Fields on the Connect form	How the merchant gets them (shown in Help)
WooCommerce	Store URL Consumer key (`ck_…`) Consumer secret (`cs_…`)	WordPress admin → **WooCommerce** → **Settings** → **Advanced** → **REST API** → **Add key**. Description "Shopify migration", Permissions **Read**, Generate. Copy both keys. Better UX (phase 1.5): WooCommerce's `/wc-auth/v1/authorize` flow — merchant clicks "Connect", approves on their site, and WooCommerce posts the keys to our callback. No copy-pasting.
WordPress	Site URL Username (optional) Application password (optional)	Public pages and posts need only the URL. For drafts/private content: WordPress admin → **Users** → **Profile** → **Application Passwords**, name it, Add, copy the password.
CSV	File upload + data type	Download the template from the app, fill it, upload. Validated before the migration begins.
Magento 2	Store URL Access token	Admin → **System** → **Extensions** → **Integrations** → **Add New**, give read access to catalog, customers, sales; Activate; copy the Access Token.
BigCommerce	Store hash Access token	Settings → **API** → **Store-level API accounts** → **Create**, read-only scopes for products, customers, orders, content.

7.1 Security of the keys

- Stored **encrypted** (AES-256-GCM, the same `utils/crypto.js` already used for Shopify tokens) in `source_connections.credentials`. Never written to logs.
- Never sent back to the browser — the form shows a masked value (`ck_4f9a…c21e`) and a "Replace keys" button.
- Only **read** access is ever asked for. The app never writes to the old store.
- Store URL must be `https://`; private and local addresses (127.0.0.1, 10.x, 192.168.x, localhost) are refused, so the server cannot be pointed at internal networks.
- Deleted on `app/uninstalled` (after a grace period) and immediately on `shop/redact` (cascade).
- A "Disconnect" button deletes the credentials at any time.

8. Field mapping — WooCommerce to Shopify

8.1 Products

WooCommerce	Shopify	Notes
`name`	`title`	
`slug`	`handle`	Kept so redirects map cleanly
`description`	`descriptionHtml`	WordPress shortcodes stripped; image URLs kept
`status` publish / draft / private	`status` ACTIVE / DRAFT / DRAFT	Or "All Draft" option
`type` simple	One default variant	
`type` variable + `variations`	`productOptions` + `variants`	Max 3 options in Shopify — else skip & report
`type` grouped / external	—	Reported as skipped (no Shopify equivalent)
`sku`, `regular_price`, `sale_price`	`sku`, `compareAtPrice`, `price`	On sale: price = sale_price, compareAt = regular_price
`stock_quantity`, `manage_stock`	`inventoryQuantities`, `inventoryItem.tracked`	At the store's primary location
`weight` (+ store unit)	`inventoryItem.measurement.weight`	Unit converted
`images[]`	`files` / media (`originalSource` URL)	First image = featured; variant images linked
`categories[]`	Collection membership	Via `id_map` (collections step runs first)
`tags[]`	`tags`	
Yoast / RankMath SEO title & description	`seo.title`, `seo.description`	Read from `meta_data` if present

8.2 Customers

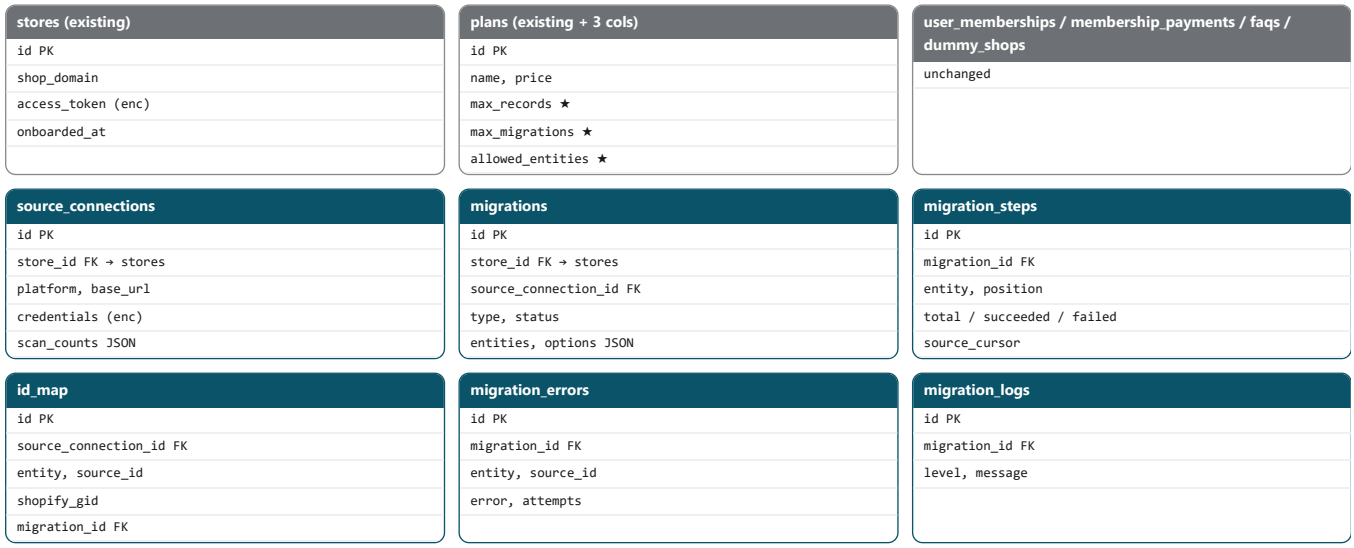
WooCommerce	Shopify	Notes
`first_name`, `last_name`, `email`	same	Email is the duplicate key
`billing.phone`	`phone`	Normalised to E.164; dropped if invalid
`billing`, `shipping`	`addresses[]`	Shipping becomes the default address
Marketing opt-in (if a plugin stores it)	`emailMarketingConsent`	Never opt someone in without a record
Password hash	—	**Cannot be migrated** — account invite email instead

8.3 Orders

WooCommerce	Shopify (`orderCreate`)	Notes
`number`	`name` (e.g. #1001)	If "keep order numbers" is on
`date_created`	`processedAt`	Original date kept
`customer_id`	`customer` (Shopify GID)	Via `id_map`; guest orders use email only
`line_items[]`	`lineItems[]` with `variantId`	Via `id_map`; unknown products become custom line items (title + price)
`shipping_lines`, `tax_lines`, `coupon_lines`	`shippingLines`, `taxLines`, `discountCode`	Totals must match the source
`status`	`financialStatus` + fulfilment	Mapping table set in Step 4
`customer_note`	`note`	
—	`options.sendReceipt: false`, `inventoryBehaviour: BYPASS`	No emails to customers, stock unchanged (unless option set)

9. Database structure

Existing tables from the skeleton are kept unchanged (grey). Six new tables are added (blue). Every new table cascades on delete from `stores`, so `shop/redact` removes everything a store owns.



Relations: stores 1—N source_connections 1—N migrations 1—N migration_steps / migration_errors / migration_logs. id_map belongs to a source_connection (so a delta run can find records from earlier runs) and records which migration created each row.

9.1 `source_connections` — the merchant's old store

Column	Type	Description
id	INT PK AI	
store_id	INT FK → stores	Shopify store that owns it. ON DELETE CASCADE
platform	ENUM('woocommerce','wordpress','csv','magento','bigcommerce')	Which adapter to use
base_url	VARCHAR(255)	https://myshop.com (normalised, no trailing slash)
credentials	TEXT	Encrypted JSON of the keys
credentials_hint	VARCHAR(64)	Masked display value, e.g. `ck_4f9a…c21e`
status	ENUM('connected','failed','disconnected')	Result of the last test
last_error	VARCHAR(512)	Readable reason when failed
source_version	VARCHAR(64)	e.g. "WooCommerce 9.3"
scan_counts	JSON	`{"products":1240,"customers":800,…}`
last_tested_at, created_at, updated_at	DATETIME / TIMESTAMP	

9.2 `migrations` — one migration run

Column	Type	Description
id	INT PK AI	Shown to the merchant as "Migration #12"
store_id	INT FK → stores	ON DELETE CASCADE
source_connection_id	INT FK → source_connections	ON DELETE CASCADE
type	ENUM('demo','full','delta','retry')	demo = 20 per data type, free
status	ENUM('draft','queued','running','paused','completed','failed','cancelled')	See lifecycle (6.1)
wizard_step	TINYINT	1–5, so a draft re-opens where it was left
entities	JSON	`["collections","products","customers","orders","redirects"]`
options	JSON	All Step 4 choices + filters (order date range)
total_records	INT	Sum of step totals at start (for plan check and %)
parent_migration_id	INT NULL FK → migrations	For retry / delta runs
worker_id, locked_at	VARCHAR(64), DATETIME	Job lock + heartbeat (6.3)
last_error	VARCHAR(512)	Fatal error when status = failed
started_at, finished_at, created_at, updated_at	DATETIME / TIMESTAMP	

9.3 `migration_steps` — progress per data type

Column	Type	Description
id	INT PK AI	
migration_id	INT FK → migrations	ON DELETE CASCADE
entity	VARCHAR(32)	collections / products / customers / orders / coupons / pages / posts / redirects
position	TINYINT	Dependency order (6.2)
status	ENUM('pending','running','done','failed','skipped')	
total	INT	From the scan (capped at 20 for demo)
processed, succeeded, failed, skipped	INT	Live counters — drive the progress bars and dashboard cards
source_cursor	VARCHAR(255)	Where to continue in the source (page number or last id)
started_at, finished_at	DATETIME	
UNIQUE (migration_id, entity)		

9.4 `id_map` — source record ↔ Shopify record

The most important table. It prevents duplicates, lets orders link to products and customers, makes resume and delta runs possible, and allows demo data to be deleted.

Column	Type	Description
id	BIGINT PK AI	
store_id	INT FK → stores	ON DELETE CASCADE
source_connection_id	INT FK → source_connections	ON DELETE CASCADE
entity	VARCHAR(32)	product / variant / collection / customer / order / …
source_id	VARCHAR(64)	ID in the old store
source_url	VARCHAR(512)	Old permalink — used to build 301 redirects
shopify_gid	VARCHAR(128)	e.g. `gid://shopify/Product/8123456789`
shopify_handle	VARCHAR(255)	New URL handle — target of the redirect
source_hash	CHAR(40)	SHA-1 of the source record; a delta run updates only when it changed
migration_id	INT FK → migrations	Run that created it (for "Delete demo data")
created_at, updated_at	TIMESTAMP	
UNIQUE (source_connection_id, entity, source_id) · INDEX (migration_id) · INDEX (shopify_gid)		

9.5 `migration_errors` — failed and skipped records

Column	Type	Description
id	BIGINT PK AI	
migration_id	INT FK → migrations	ON DELETE CASCADE
entity, source_id	VARCHAR(32), VARCHAR(64)	Which record
source_name	VARCHAR(255)	Product title / customer email — readable in the report
kind	ENUM('failed','skipped')	Skipped = deliberately not migrated (duplicate, unsupported)
error	TEXT	Readable reason (Shopify `userErrors` translated)
attempts	TINYINT	Incremented on each retry
resolved	TINYINT(1)	1 once a retry succeeds
created_at, updated_at	TIMESTAMP	
UNIQUE (migration_id, entity, source_id) · INDEX (migration_id, resolved)		

9.6 `migration_logs` — timeline for support

Column	Type	Description
id	BIGINT PK AI	
migration_id	INT FK → migrations	ON DELETE CASCADE
level	ENUM('info','warn','error')	
message	VARCHAR(1024)	"Products step started", "Source returned 503, retrying"…
created_at	TIMESTAMP	Rows older than 90 days are purged

9.7 Changes to existing `plans`

Column	Type	Description

max_records	INT NULL	Records per billing month across all migrations. NULL = unlimited
max_migrations	INT NULL	Full migrations per month. NULL = unlimited
allowed_entities	JSON NULL	Data types the plan allows, e.g. Free: products only. NULL = all

10. Shopify requirements

10.1 Access scopes (`SHOPIFY_SCOPES`)

Scope	Needed for
`write_products`	Products, variants, collections
`write_inventory` , `read_locations`	Stock levels at a location
`write_files`	Product images and media
`write_publications`	Publishing to the Online Store
`write_customers`	Customers, addresses, account invites
`write_orders`	Historical orders
`write_discounts`	Coupons → discount codes
`write_content`	Pages and blog articles
`write_online_store_navigation`	301 URL redirects

> **Protected customer data.** Creating customers and orders requires approval for **protected customer data** in the Partner Dashboard (App → API access → Protected customer data), including the name, email, phone and address fields (Level 2). Without it those API calls are refused. Apply early — Shopify reviews the request, and the app's privacy policy must explain what is stored and for how long.

10.2 Shopify APIs used

Data	GraphQL Admin API	Notes
Collections	`collectionCreate`	Custom collections
Products	`productSet`	Product, options, variants, media and inventory in one call; idempotent per handle
Publishing	`publishablePublish`	To the Online Store publication
Customers	`customerCreate` , `customerSendAccountInviteEmail`	
Orders	`orderCreate`	With `sendReceipt: false` and inventory bypass
Coupons	`discountCodeBasicCreate`	
Pages, posts	`pageCreate` , `blogCreate` , `articleCreate`	
Redirects	`urlRedirectCreate`	e.g. `/product/linen-shirt/` → `/products/linen-shirt`
Large volume	`stagedUploadsCreate` + `bulkOperationRunMutation`	10,000+ records
Billing	`appSubscriptionCreate`	Already built

10.3 Webhooks

Topic	Action
`app/uninstalled`	Existing. Also: cancel running migrations, schedule credential deletion
`shop/redact`	Existing. Cascade deletes everything incl. credentials and id_map
`customers/redact` , `customers/data_request`	Existing. Look up the customer in `id_map` ; we store only IDs, no personal data

10.4 Server environment (.env additions)

```
SHOPIFY_SCOPES=write_products,write_inventory,read_locations,write_files,write_publications,write_customers,write_orders,write_discounts,write_content,write_online_
MIGRATION_WORKER_ENABLED=true        # run the worker loop in this process
MIGRATION_WORKER_INTERVAL_MS=3000    # how often to look for work
MIGRATION_CONCURRENCY=2              # migrations processed at the same time
SOURCE_PAGE_SIZE=50                  # records fetched per source request
SOURCE_TIMEOUT_MS=30000
DEMO_RECORDS_PER_ENTITY=20
```

11. Routes and endpoints

All routes are behind `requireSession` and scoped to `req.storeId` — a merchant can only see and act on their own migrations.

Method	Path	Purpose
GET	`/dashboard`	Dashboard with cards (section 3)
GET	`/migrations`	All migrations, filter by status, paged
GET	`/migrations/new`	Wizard — creates or resumes a draft
GET	`/migrations/:id/wizard/:step`	Wizard step 1–5 for a draft
POST	`/connections/test`	Validate URL + keys, return version and counts (JSON)
POST	`/connections`	Save a tested connection (encrypts keys)
POST	`/connections/:id/delete`	Disconnect and delete the keys
POST	`/migrations/:id/wizard`	Save a wizard step (platform, connection, entities, options)
POST	`/migrations/:id/start`	Plan check, create steps, status → queued. Body: `{ type: "demo" | "full" }`
GET	`/migrations/:id`	Progress or results page
GET	`/migrations/:id/status`	JSON for polling: status, steps, counters, ETA
POST	`/migrations/:id/pause` · `/resume` · `/cancel`	Control a running migration
POST	`/migrations/:id/retry`	New migration of type retry for unresolved errors
POST	`/migrations/:id/delta`	New migration of type delta (recent data)
POST	`/migrations/:id/delete-demo-data`	Remove demo records from Shopify
GET	`/migrations/:id/errors.csv`	Download failed / skipped records

12. Project file structure

New files (★) added to the existing skeleton:

```
config/
  migrate.js                      + 6 new tables, 3 plan columns
controllers/
  dashboardController.js          + card queries
  migrationController.js      ★   list, wizard, start, status, pause, retry, csv
  connectionController.js     ★   test, save, delete
models/
  sourceConnectionModel.js    ★
  migrationModel.js           ★   incl. dashboard stats, lock/claim
  migrationStepModel.js       ★
  idMapModel.js               ★
  migrationErrorModel.js      ★
routes/
  migrationRoute.js           ★
  connectionRoute.js          ★
services/
  migrationWorker.js          ★   the loop: claim → step → page → records → save
  sources/
    index.js                  ★   platform → adapter registry
    woocommerce.js            ★
    wordpress.js              ★
    csv.js                    ★   (phase 2)
  destinations/
    shopify.js                ★   createProduct / createCustomer / createOrder …
  transform/
    html.js                   ★   strip shortcodes, fix image URLs
    address.js                ★   country/province codes, phone E.164
  safeUrl.js                  ★   https only, blocks private/local addresses
views/
  dashboard.ejs                   + cards
  migrations/
    list.ejs                  ★
    wizard.ejs                ★   steps 1–5
    progress.ejs              ★
    results.ejs               ★
tests/
  woocommerceTransform.test.js ★  sample Woo JSON → expected Shopify input
  migrationWorker.test.js     ★   resume, duplicate protection, error isolation
```

13. Plans, limits and demo migration

Plan	Price / month	Records / month	Data types	Extras
Free	$0	Demo only (20 per type)	All (demo)	Community support
Starter	$19	2,000	Products, collections, customers	Email support
Pro	$49	25,000	All	Delta migration, redirects, priority support
Unlimited	$99	Unlimited	All	Bulk operations, migration assistance

Prices are placeholders for the business to decide. Limits are enforced from the plan columns (9.7), never from the card text.

- The plan check runs on **Step 5** (before begin) using the scanned totals, and again in the worker before each page so a downgrade mid-month is respected.
- **Demo migrations** never count towards the limit. They are marked in `id_map.migration_id` so they can be deleted with one click.
- Many merchants migrate once. A **one-time charge** (`appPurchaseOneTimeCreate`) priced by record count is a sensible alternative or add-on to monthly plans.

14. What cannot be migrated

State these in Help & support and on Step 3, so expectations are right before anyone pays:

Item	Why / what we do instead
Customer passwords	Stored as one-way hashes Shopify cannot read. Customers get an account invite email to set a new password.
Theme and design	Shopify themes are a different system. The merchant picks a Shopify theme.
Payment gateways, tax rules, shipping zones	Store settings, configured in Shopify admin. Help lists where.
WordPress plugins and their data	Only well-known data is read (e.g. Yoast SEO). Others need a custom job.
Products with more than 3 attributes	Shopify allows 3 options per product. Skipped and reported, or extra attributes merged (option).
Grouped and external/affiliate products	No Shopify equivalent. Reported as skipped.
Subscriptions and memberships	Need a Shopify subscription app.
Images behind a login or firewall	Shopify must be able to download them from a public URL.

15. Build phases

Phase	Deliverable	Includes
1	Foundation	6 new tables, nav, Migrations list, wizard Steps 1–2, WooCommerce connect + test + scan, key encryption and URL safety
2	First data moves	Worker loop with locking and resume, `id_map`, collections + products, progress screen
3	Full WooCommerce	Customers, orders, coupons, redirects; wizard Steps 3–5; results screen, error CSV, retry
4	Dashboard and plans	Dashboard cards, demo migration + delete demo data, plan limits, setup steps, Help content
5	WordPress content	Pages, posts, delta migration
6	More sources	CSV import with templates, then Magento and BigCommerce adapters; bulk operations for large stores
7	App Store launch	Protected customer data approval, privacy policy, listing, review submission

Appendix A — Full SQL for the new tables

```
CREATE TABLE source_connections (
  id               INT AUTO_INCREMENT PRIMARY KEY,
  store_id         INT NOT NULL,
  platform         ENUM('woocommerce','wordpress','csv','magento','bigcommerce') NOT NULL,
  base_url         VARCHAR(255) NOT NULL,
  credentials      TEXT NULL,                 -- AES-256-GCM encrypted JSON
  credentials_hint VARCHAR(64) NULL,
  status           ENUM('connected','failed','disconnected') NOT NULL DEFAULT 'connected',
  last_error       VARCHAR(512) NULL,
  source_version   VARCHAR(64) NULL,
  scan_counts      JSON NULL,
  last_tested_at   DATETIME NULL,
  created_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at       TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY idx_conn_store (store_id, status),
  CONSTRAINT fk_conn_store FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE migrations (
  id                   INT AUTO_INCREMENT PRIMARY KEY,
  store_id             INT NOT NULL,
  source_connection_id INT NULL,
  type                 ENUM('demo','full','delta','retry') NOT NULL DEFAULT 'full',
  status               ENUM('draft','queued','running','paused','completed','failed','cancelled')
                       NOT NULL DEFAULT 'draft',
  wizard_step          TINYINT NOT NULL DEFAULT 1,
  entities             JSON NULL,
  options              JSON NULL,
  total_records        INT NOT NULL DEFAULT 0,
  parent_migration_id  INT NULL,
  worker_id            VARCHAR(64) NULL,
  locked_at            DATETIME NULL,
  last_error           VARCHAR(512) NULL,
  started_at           DATETIME NULL,
  finished_at          DATETIME NULL,
  created_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  KEY idx_mig_store_status (store_id, status),
  KEY idx_mig_work (status, locked_at),
  CONSTRAINT fk_mig_store  FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE CASCADE,
  CONSTRAINT fk_mig_conn   FOREIGN KEY (source_connection_id) REFERENCES source_connections(id) ON DELETE CASCADE,
  CONSTRAINT fk_mig_parent FOREIGN KEY (parent_migration_id) REFERENCES migrations(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE migration_steps (
  id           INT AUTO_INCREMENT PRIMARY KEY,
  migration_id INT NOT NULL,
  entity       VARCHAR(32) NOT NULL,
  position     TINYINT NOT NULL,
  status       ENUM('pending','running','done','failed','skipped') NOT NULL DEFAULT 'pending',
  total        INT NOT NULL DEFAULT 0,
  processed    INT NOT NULL DEFAULT 0,
  succeeded    INT NOT NULL DEFAULT 0,
  failed       INT NOT NULL DEFAULT 0,
  skipped      INT NOT NULL DEFAULT 0,
  source_cursor VARCHAR(255) NULL,
  started_at   DATETIME NULL,
  finished_at  DATETIME NULL,
  UNIQUE KEY uniq_step (migration_id, entity),
  CONSTRAINT fk_step_mig FOREIGN KEY (migration_id) REFERENCES migrations(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE id_map (
  id                   BIGINT AUTO_INCREMENT PRIMARY KEY,
  store_id             INT NOT NULL,
  source_connection_id INT NOT NULL,
  entity               VARCHAR(32) NOT NULL,
  source_id            VARCHAR(64) NOT NULL,
  source_url           VARCHAR(512) NULL,
  shopify_gid          VARCHAR(128) NOT NULL,
  shopify_handle       VARCHAR(255) NULL,
  source_hash          CHAR(40) NULL,
  migration_id         INT NULL,
  created_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at           TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_map (source_connection_id, entity, source_id),
  KEY idx_map_migration (migration_id),
  KEY idx_map_gid (shopify_gid),
  CONSTRAINT fk_map_store FOREIGN KEY (store_id) REFERENCES stores(id) ON DELETE CASCADE,
  CONSTRAINT fk_map_conn  FOREIGN KEY (source_connection_id) REFERENCES source_connections(id) ON DELETE CASCADE,
  CONSTRAINT fk_map_mig   FOREIGN KEY (migration_id) REFERENCES migrations(id) ON DELETE SET NULL
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE migration_errors (
  id           BIGINT AUTO_INCREMENT PRIMARY KEY,
  migration_id INT NOT NULL,
  entity       VARCHAR(32) NOT NULL,
  source_id    VARCHAR(64) NOT NULL,
  source_name  VARCHAR(255) NULL,
  kind         ENUM('failed','skipped') NOT NULL DEFAULT 'failed',
  error        TEXT NULL,
  attempts     TINYINT NOT NULL DEFAULT 1,
  resolved     TINYINT(1) NOT NULL DEFAULT 0,
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
```

```
  updated_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  UNIQUE KEY uniq_err (migration_id, entity, source_id),
  KEY idx_err_open (migration_id, resolved),
  CONSTRAINT fk_err_mig FOREIGN KEY (migration_id) REFERENCES migrations(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

CREATE TABLE migration_logs (
  id           BIGINT AUTO_INCREMENT PRIMARY KEY,
  migration_id INT NOT NULL,
  level        ENUM('info','warn','error') NOT NULL DEFAULT 'info',
  message      VARCHAR(1024) NOT NULL,
  created_at   TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP,
  KEY idx_log_mig (migration_id, created_at),
  CONSTRAINT fk_log_mig FOREIGN KEY (migration_id) REFERENCES migrations(id) ON DELETE CASCADE
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;

ALTER TABLE plans
  ADD COLUMN max_records      INT  NULL DEFAULT NULL,
  ADD COLUMN max_migrations   INT  NULL DEFAULT NULL,
  ADD COLUMN allowed_entities JSON NULL DEFAULT NULL;
```